LAB_4 - Recursive Algorithms

A2 - S4 - 2025/2026 - II.2415

Advanced Algorithms and Programming

Team 04

Damiano Spadaccini - Nicolò Romanelli

GitHub repository: https://github.com/damspada/AAP-T04-Semester-Project (branch LAB_04_Recursive_Algorithms, folder LAB_04)

This is the final version of our LAB_4 report. We kept the pseudo-code and the answers of the report written during the session (T04_LAB4_AES). During the session we did not finish everything, so we added the missing functions and the missing answers. For each exercise we also added more complexity analysis, a short reflection and the tests of our Python code, and at the end the final question.

The code is on GitHub (link above). Each file can be run alone and prints the result of every test.

EXERCISE 1 - RECURSIVE COMMENT THREAD TRAVERSAL

A thread is a tree: every comment has a list of replies, and every reply is a comment too. NULL is the empty pointer.

```
Structure CommentNode
    comment_id: Integer
    user_id: Integer
    content: String
    timestamp: Integer
    likes: Integer
    replies: List of pointer to CommentNode
End Structure
```

```
Procedure DisplayThread(c: pointer to CommentNode, level: Integer)
    If c != NULL Then
        // 2 spaces for each level
        Print(Spaces(2 * level), c.user_id, ": ", c.content)
        // level + 1 for the replies, so the indentation comes for free
        For each r in c.replies Do
            DisplayThread(r, level + 1)
        End For
    End If
End
```

```
Function CountTotalComments(c: pointer to CommentNode) -> Integer
    total <- 0
    If c != NULL Then
        // the comment itself and all the comments in its replies
        total <- 1
        // then we just ask every reply how many comments it has
        For each r in c.replies Do
            total <- total + CountTotalComments(r)
        End For
    End If
    Return total
End
```

```
Function TotalLikes(c: pointer to CommentNode) -> Integer
    likes <- 0
    If c != NULL Then
        likes <- c.likes
        For each r in c.replies Do
            likes <- likes + TotalLikes(r)
        End For
    End If
    Return likes
End

// c must not be NULL, a comment without replies has depth 0
Function FindDeepestReply(c: pointer to CommentNode) -> Integer
    deepest <- 0
    For each r in c.replies Do
        d <- 1 + FindDeepestReply(r)
        // keep the biggest one we saw until now
        If d > deepest Then
            deepest <- d
        End If
    End For
    Return deepest
End

Function ContainsKeyword(keyword: String, c: pointer to CommentNode) -> Boolean
    found <- False
    If c != NULL Then
        If Contains(c.content, keyword) Then
            found <- True
        End If
        i <- 1
        // stop as soon as the keyword is found
        While Not found And i <= Length(c.replies) Do
            found <- ContainsKeyword(keyword, c.replies[i])
            i <- i + 1
        End While
    End If
    Return found
End
```

Added after the session: SearchByUser and DeleteComment.

```
Function SearchByUser(user_id: Integer, c: pointer to CommentNode) -> List of pointer to CommentNode
    result <- empty list
    CollectByUser(user_id, c, result)
    Return result
End

// all the comments go in the same list, so no list is copied
Procedure CollectByUser(user_id: Integer, c: pointer to CommentNode, output result: List of pointer to
    CommentNode)
    If c != NULL Then
        If c.user_id = user_id Then
            Append(result, c)
        End If
        For each r in c.replies Do
            CollectByUser(user_id, r, result)
        End For
    End If
End
```

```
// returns the thread without the comment and all its replies (NULL if the first comment is deleted)
Function DeleteComment(comment_id: Integer, thread: pointer to CommentNode) -> pointer to
    CommentNode
    If thread = NULL Then
        Return NULL
    End If
    If thread.comment_id = comment_id Then
        Return NULL
    End If
    new_replies <- empty list
    For each r in thread.replies Do
        kept <- DeleteComment(comment_id, r)
        If kept != NULL Then
            Append(new_replies, kept)
        End If
    End For
    thread.replies <- new_replies
    Return thread
End
```

1. What is the time complexity of traversing an entire comment thread with n total comments?

O(n), because every comment is visited exactly one time and each visit does a constant amount of work.

2. How does recursion depth relate to the comment nesting structure? What happens with very deep threads (1000+ levels)?

The recursion depth is equal to the nesting depth of the thread: there is one open call for each level, so a thread with depth d has d + 1 calls open at the same time. With 1000+ levels the call stack becomes too big and we get a stack overflow.

3. Compare memory usage between recursive and iterative stack-based implementations.

The recursive version needs one frame for each open call, so O(d). The iterative version uses its own stack: it can hold up to O(n) comments, but it can not overflow like the call stack.

More complexity analysis and reflection

All the functions visit each comment one time, so they are O(n). The recursion uses O(d) memory. ContainsKeyword stops when it finds the keyword.

Test set

The tests use the thread of the lab example (likes 10, 5, 2, 7, 3 and 1). All the tests print OK.

Case	**Operations**	**Expected**	**Got**
lab example	DisplayThread: line of 401 (level 3)	6 spaces + "Bob: It was delicious!"	6 spaces + "Bob: It was delicious!"
lab example	CountTotalComments	6	6
lab example	TotalLikes	28	28
lab example	FindDeepestReply	3	3
lab example	SearchByUser(Alice)	101, 301, 302	101, 301, 302
lab example	ContainsKeyword("olive"), ("delicious")	True, True	True, True
user who never commented	SearchByUser(Dave)	empty list	empty list
keyword not in the thread	ContainsKeyword("pizza")	False	False
upper and lower case	ContainsKeyword("Olive")	False	False
empty thread	CountTotalComments(NULL), TotalLikes(NULL)	0, 0	0, 0
single comment	CountTotalComments, FindDeepestReply	1, 0	1, 0
content too long	comment of 150 characters	100 characters kept	100 characters kept
delete with cascade	DeleteComment(201)	101, 202, 302 left, depth 2	101, 202, 302 left, depth 2

Case	Operations	Expected	Got
comment that does not exist	DeleteComment(999)	nothing changes (3 comments)	3 comments
delete a leaf	DeleteComment(302)	2 comments	2 comments
delete the first comment	DeleteComment(101)	NULL	NULL
empty thread	DeleteComment(1, NULL)	NULL	NULL

EXERCISE 2 - RECURSIVE CONTENT AGGREGATION WITH DIVIDE & CONQUER

posts is an array of Post. Every function works on the part posts[left .. right] and splits it in two halves.

```
Structure Post
    post_id: Integer
    user_id: Integer
    content_preview: String
    timestamp: Integer
    likes: Integer
    comments: Integer
    shares: Integer
    engagement_score: Integer
End Structure

Function Score(p: Post) -> Integer
    Return p.likes * 1 + p.comments * 2 + p.shares * 3
End

// returns the position of the post with the highest score, so we know the post and its score
Function MaxEngagement(posts: Array of Post, left: Integer, right: Integer) -> Integer
    If left = right Then
        // only one post, so it is the max for sure
        Return left
    End If
    mid <- (left + right) div 2
    a <- MaxEngagement(posts, left, mid)
    b <- MaxEngagement(posts, mid + 1, right)
    If posts[a].engagement_score >= posts[b].engagement_score Then
        best <- a
    Else
        best <- b
    End If
    Return best
End

Function SumEngagement(posts: Array of Post, left: Integer, right: Integer) -> Integer
    If left > right Then
        // empty part
        Return 0
    End If
    If left = right Then
        Return posts[left].engagement_score
    End If
    // same idea as merge sort: split, solve the two halves, put them together
    mid <- (left + right) div 2
    Return SumEngagement(posts, left, mid) + SumEngagement(posts, mid + 1, right)
End

Function AverageEngagement(posts: Array of Post, left: Integer, right: Integer) -> Real
    n <- right - left + 1
    average <- 0
    If n > 0 Then
        average <- SumEngagement(posts, left, right) / n
    End If
    Return average
End
```

```
Function CountAboveThreshold(posts: Array of Post, left: Integer, right: Integer, threshold: Integer) ->
    Integer
    If left > right Then
        Return 0
    End If
    If left = right Then
        count <- 0
        If posts[left].engagement_score > threshold Then
            count <- 1
        End If
        Return count
    End If
    mid <- (left + right) div 2
    Return CountAboveThreshold(posts, left, mid, threshold) + CountAboveThreshold(posts, mid + 1,
        right, threshold)
End

// the posts are sorted from the highest to the lowest score
Procedure MergeSortByEngagement(output posts: Array of Post, left: Integer, right: Integer)
    If left < right Then
        mid <- (left + right) div 2
        MergeSortByEngagement(posts, left, mid)
        MergeSortByEngagement(posts, mid + 1, right)
        Merge(posts, left, mid, right)
    End If
End

// basically the Merge of the slides, but with posts and the biggest score first
Procedure Merge(output posts: Array of Post, left: Integer, mid: Integer, right: Integer)
    i <- left
    j <- mid + 1
    k <- left
    While i <= mid And j <= right Do
        // with ">=" two posts with the same score keep their order
        If posts[i].engagement_score >= posts[j].engagement_score Then
            Temp[k] <- posts[i]
            i <- i + 1
        Else
            Temp[k] <- posts[j]
            j <- j + 1
        End If
        k <- k + 1
    End While
    // copy the rest of the half that is not finished
    While i <= mid Do
        Temp[k] <- posts[i]
        i <- i + 1
        k <- k + 1
    End While
    While j <= right Do
        Temp[k] <- posts[j]
        j <- j + 1
        k <- k + 1
    End While
    For k <- left To right Do
        posts[k] <- Temp[k]
    End For
End
```

Added after the session: FindPeakHour.

```
// the likes must increase until the peak and then decrease
Function FindPeakHour(likes: Array of Integer, left: Integer, right: Integer) -> Integer
    If left = right Then
        Return left
    End If
    mid <- (left + right) div 2
    If likes[mid] < likes[mid + 1] Then
        // likes[mid + 1] is bigger, so the peak is on the right
        peak <- FindPeakHour(likes, mid + 1, right)
    Else
        // otherwise the peak is mid or on the left
        peak <- FindPeakHour(likes, left, mid)
    End If
    Return peak
End
```

1. What is the time complexity of recursive max_engagement? Prove it.

O(n). Every call does one comparison and makes two calls on the two halves, so T(n) = 2T(n/2) + O(1) with T(1) = O(1). The recursion tree has 1 + 2 + 4 + ... + n = 2n - 1 calls, each with O(1) work, so T(n) = O(n).

2. Compare recursive merge sort (O(n log n)) with iterative insertion sort (O(n^2)) for sorting 10,000 posts.

Merge sort does about n log2(n) = 10 000 * 14 = 140 000 comparisons (our code counted 120 467). Insertion sort does about n^2 / 4 = 25 000 000 comparisons on random posts, and n^2 / 2 in the worst case, so about 200 times more.

3. What is the recursion depth for merge sort on n elements?

The depth is ceil(log2 n), because the array is split in half at every level of the recursion.

4. For find_peak_hour, why can we use binary search-style recursion? What property must the array satisfy?

Because at each step we know which half has the peak: if likes[mid] < likes[mid + 1], the peak is on the right, otherwise it is mid or on the left. So we keep only one half, like in binary search. The array must increase until the peak and then decrease (only one peak).

More complexity analysis and reflection

MaxEngagement, SumEngagement and CountAboveThreshold are O(n), and the recursion is O(log n) deep. MergeSortByEngagement is O(n log n) in every case and uses O(n) memory for Temp. FindPeakHour is O(log n).

Test set

The posts of the lab example have the scores 150, 320, 95 and 280. All the tests print OK.

Case	**Operations**	**Expected**	**Got**
lab example	MaxEngagement(posts, 0, 3)	Post2 (320)	Post2 (320)
lab example	SumEngagement, AverageEngagement	845, 211.25	845, 211.25
lab example	CountAboveThreshold(200)	2	2
lab example	MergeSortByEngagement	Post2, Post4, Post1, Post3	Post2, Post4, Post1, Post3
single post	MaxEngagement(posts, 2, 2)	position 2	position 2
empty part	SumEngagement(posts, 3, 2), AverageEngagement	0, 0	0, 0
threshold equal to a score	CountAboveThreshold(320)	0 (strictly greater)	0
negative threshold	CountAboveThreshold(-1)	4	4
equal scores	MaxEngagement, MergeSort	the first post, same order	the first post, same order
200 random posts	MergeSortByEngagement	sorted	sorted

Case	Operations	Expected	Got
lab example	FindPeakHour (24 hours)	hour 4 (30 likes)	hour 4
likes only increase / only decrease	FindPeakHour	hour 23 / hour 0	hour 23 / hour 0
only one hour	FindPeakHour([42], 0, 0)	hour 0	hour 0
two peaks	FindPeakHour([1, 9, 2, 3, 4, 5, 6, 7])	a local peak (hour 7)	hour 7

EXERCISE 3 - CONVERTING RECURSION TO ITERATION WITH EXPLICIT STACK

The list is in depth-first order: a comment, then all its replies (with their replies), then the next reply.

```
Function FlattenRecursive(c: pointer to CommentNode) -> List of pointer to CommentNode
    result <- empty list
    AddToList(c, result)
    Return result
End

Procedure AddToList(c: pointer to CommentNode, output result: List of pointer to CommentNode)
    If c != NULL Then
        Append(result, c)
        For each r in c.replies Do
            AddToList(r, result)
        End For
    End If
End

// the stack contains pairs (comment, state)
// STATE_START: just pushed, the replies are not processed yet
// STATE_REPLIES_DONE: all the replies are processed
Function FlattenIterative(c: pointer to CommentNode) -> List of pointer to CommentNode
    result <- empty list
    If c != NULL Then
        S <- empty stack
        Push(S, (c, STATE_START))
        While Not IsEmpty(S) Do
            (x, state) <- Pop(S)
            // first time we see x: we add it and then we go to its replies
            If state = STATE_START Then
                Append(result, x)
                Push(S, (x, STATE_REPLIES_DONE))
                // replies in reverse order, so the first reply is popped first
                For i <- Length(x.replies) DownTo 1 Do
                    Push(S, (x.replies[i], STATE_START))
                End For
            End If
            // with STATE_REPLIES_DONE there is nothing more to do for x
        End While
    End If
    Return result
End
```

Added after the session: CountCommentsTail and CountCommentsLoop.

```
// pending = comments still to count; the recursive call is the last thing the function does
// first call: CountCommentsTail(stack with only the first comment, 0)
Function CountCommentsTail(pending: Stack of pointer to CommentNode, accumulator: Integer) ->
    Integer
    If IsEmpty(pending) Then
        Return accumulator
    End If
    c <- Pop(pending)
    For each r in c.replies Do
        Push(pending, r)
    End For
    Return CountCommentsTail(pending, accumulator + 1)
End
```

```
// the same function, the tail call becomes a new turn of the loop
Function CountCommentsLoop(comment: pointer to CommentNode) -> Integer
    pending <- empty stack
    Push(pending, comment)
    accumulator <- 0
    While Not IsEmpty(pending) Do
        c <- Pop(pending)
        For each r in c.replies Do
            Push(pending, r)
        End For
        accumulator <- accumulator + 1
    End While
    Return accumulator
End
```

1. For a thread with depth d and total n comments, compare maximum stack size needed for the recursive implementation and the iterative with explicit stack.

Recursive: d + 1 calls on the call stack, so O(d). Iterative: the stack contains the comments of the current path and the replies that are waiting, so in the worst case (one comment with n - 1 replies) it contains about n pairs, O(n).

2. When would you prefer the recursive version? When iterative?

The recursive version when the depth is small, because the code is shorter and simpler. The iterative version when the depth can be very big, because it can not cause a stack overflow.

3. How does tail recursion conversion reduce stack usage? Can all recursive functions be made tail recursive?

The recursive call is the last thing the function does, so its frame can be used again for the next call and the stack stays O(1). No: if the function does something after the recursive call, it is not tail recursive. We have to change it with an accumulator, and for a tree also with a stack of the comments to visit.

4. Explain the state machine approach - why is it necessary for functions that process nested structures?

With our own stack we do not know anymore where each call was, so we save it in a state. START: we just found the comment, we add it and push its replies. REPLIES_DONE: we finished its replies. We need it because with nested comments some work must be done after the replies of a comment.

5. In a production social network with user-generated nested comments, which approach would you choose and why?

The iterative version with an explicit stack, because users can create very deep threads and a stack overflow would stop the server.

More complexity analysis and reflection

FlattenRecursive and FlattenIterative are O(n). FlattenRecursive uses O(d) memory, FlattenIterative O(n) in the worst case.

Performance comparison: when every comment has one reply, the recursive version has depth + 1 open calls and our stack has the same size. Python stops at 1000 calls, so the recursive versions crash at about 1000 levels (also the tail recursive one), but the iterative versions still work.

Test set

All the tests print OK.

Case	Operations	Expected	Got
lab example	FlattenRecursive, FlattenIterative	Comment1, Reply1, SubReply1, Reply2	Comment1, Reply1, SubReply1, Reply2
lab example	CountCommentsTail, CountCommentsLoop	4, 4	4, 4
empty thread	FlattenRecursive(NULL), FlattenIterative(NULL)	empty list	empty list
single comment	FlattenIterative	the comment	the comment
one comment with 5 replies	FlattenIterative	same order as recursive, stack max 6	same order, stack max 6

Case	Operations	Expected	Got
very deep thread (2000 levels)	recursive count	RecursionError	RecursionError
very deep thread (2000 levels)	FlattenIterative, CountCommentsLoop	2001, 2001	2001, 2001
very deep thread (2000 levels)	CountCommentsTail	RecursionError (no tail call optimization)	RecursionError

Depth	Recursive calls open	Explicit stack max	Same result
5	6	6	yes
10	11	11	yes
50	51	51	yes
100	101	101	yes
500	501	501	yes
2000	crash (limit 1000)	2001	only iterative

FINAL INTEGRATION QUESTION

Based on your previous work, design a feature: "Engagement Analytics with Nested Comments", combining set operations, doubly linked lists, priority queues and recursive algorithms.

In the semester project we have to build the backend of a social network, so the feature could work like this:

- the priority queue sorts the posts by engagement, and we put the top posts in a doubly linked list to move between them;
- for each post we put the users who commented in a set, and the intersection of the sets gives the users who commented on several posts;
- we use recursion on small threads (count, likes, display) and our own stack on deep threads, so there is no stack overflow;
- for millions of posts we use a heap for the ranking, hash sets for the users, and we keep the totals of each thread in the post instead of computing them again.